EXPERIMENT 14

Title: Implementing a Packet Sniffer Using Raw Sockets

AIM

To write a Python program using raw sockets to capture network packets and display basic packet information such as source MAC address, destination MAC address, protocol, and packet type.

ALGORITHM

1. Import the socket and struct modules.
2. Create a raw socket using the Ethernet packet protocol.
3. Receive an Ethernet frame from the network interface.
4. Extract the destination MAC, source MAC, and Ethernet protocol field.
5. Identify common packet types such as IPv4, ARP, and IPv6.
6. Display the extracted packet information.
7. Close the socket when the program is interrupted.

PROGRAM (Python — Linux)

```
import socket
import struct

def mac_address(raw):
    return ":".join(f"{b:02x}" for b in raw)

# Run with administrator/root privileges on Linux.
raw_socket = socket.socket(
    socket.AF_PACKET, socket.SOCK_RAW,
    socket.ntohs(3)  # Capture all Ethernet protocols
)

try:
    while True:
        packet, address = raw_socket.recvfrom(65535)

        dest_mac = mac_address(packet[0:6])
        src_mac = mac_address(packet[6:12])
        protocol = struct.unpack("!H", packet[12:14])[0]

        if protocol == 0x0800:
            packet_type = "IPv4"
        elif protocol == 0x0806:
            packet_type = "ARP"
        elif protocol == 0x86DD:
            packet_type = "IPv6"
        else:
            packet_type = "Other"

        print("Source MAC      :", src_mac)
        print("Destination MAC :", dest_mac)
        print("Protocol        : 0x%04x" % protocol)
        print("Packet type     :", packet_type)
        print("-" * 40)

except KeyboardInterrupt:
    print("\nPacket capture stopped.")
finally:
    raw_socket.close()
```

SAMPLE OUTPUT

```
Source MAC      : 3c:52:82:11:22:23
```

```
Destination MAC : ff:ff:ff:ff:ff:ff
Protocol        : 0x0806
Packet type     : ARP
--------------------------------------
```

RESULT

Thus, a packet-sniffing program using raw sockets was implemented successfully. It captures Ethernet frames and displays the source MAC address, destination MAC address, protocol number, and packet type.

Note: This example uses Linux-specific AF_PACKET sockets and requires root/administrator privileges. Use it only on a network you own or are authorized to monitor. It will not run as written on Windows.